esprit Se former autrement HONORIS UNITED UNIVERSITIES	**EXAMEN** **Semestre** : ■ 1 ☐ 2 **Période** : ■ 1 ☐ 2 **Session** : ■ Principale ☐ Rattrapage

Module : Architecture des systèmes d'information I (.NET)
Enseignants : Équipe .NET
Classes :

Documents autorisés : ☐ OUI ■ NON
Calculatrice autorisée : ☐ OUI ■ NON
Nombre de pages : 7 pages

Date : **Heure** : **Durée** : 1h30m

Nom et Prénom : ……	**Classe :** ……..

- **Toute question à choix multiples a une réponse <u>unique</u>**
- **Si une question est à choix multiples, entourer ou cocher la bonne réponse**

Barème									
Q1	**Q2**	**Q3**	**Q4**	**Q5**	**Q6**	**Q7**	**Q8**	**Q9**	**Q10**
0.75	0.75	0.75	0.75	0.75	1	1	0.75	0.75	0.75
Q11	**Q12**	**Q13**	**Q14**	**Q15**	**Q16**	**Q17**	**Q18**	**Q19**	**Q20**
2	0.5	0.5	1	0.5	1	1	1.5	1	0.5
Q21	**Q22**	**Q23**	**Q24**						
0.5	0.75	0.5	0.75						

Partie I : Généralités (8 pts)

1) Pour quel système d'exploitation il n'existe pas de version de **.NET Core**
   a) Windows
   b) Linux
   c) Android
   d) MacOS
2) Parmi les propositions suivantes, laquelle n'est pas un type en **.NET**
   a) Classe

b) Structure

c) Pointeur

d) Délégué

3) Une classe est déclarée dans un espace de noms nommé **Security**, choisir le bon code permettant de l'utiliser dans un autre espace de noms :

☒ `using Security;`	☐ `using Security.*;`
☐ `import Security.*;`	☐ `import Security;`

4) Un type a été déclaré sans utiliser un mot clé d'encapsulation, quelle sera le mot clé retenu lors de la compilation :

☐ `default`	☒ `internal`
☐ `public`	☐ `private`

5) On a une classe générique non abstraire nommée **G<T1,T2>**, écrire un code qui déclare une variable de ce type, dans une méthode en utilisant le mot clé var :

var g = new G<int,int>()

6) On veut déclarer une méthode d'extension qui retourne une liste d'entiers, compléter le code suivant :

```
public static class Extensions
{
    public static IEnumerable<int> M(this string s)
    {
        return s.ToArray();
    }
}
```

7) En utilisant une méthode d'extensions du **LINQ**, convertit la source de données en une liste de type **string** avec le caractère **s** comme début :

```
int [] source = new int[] {1, 5, -8, 9};
source.Select(e => "s" + e);
```

8) Le package de base pour **EF Core** s'appelle :

a) **Microsoft.EntityFrameworkCore.Abstractions**

b) **Microsoft.EntityFrameworkCore**

c) **Microsoft.EntityFrameworkCore.SqlServer**

d) **Microsoft.EntityFrameworkCore.Tools**

9) Choisir le code correct pour créer la classe de contexte de données :

a) `class AppContext : DbContext`

b) `class AppContext : DBContext`

c) `class AppContext : EntityFrameworkCoreDBContext`

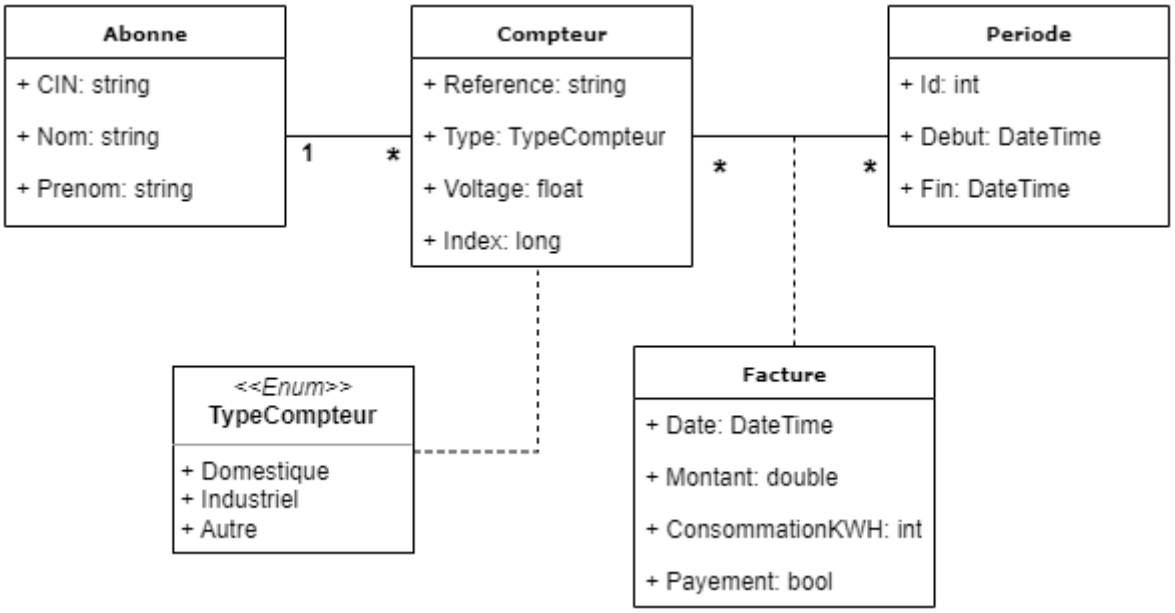
d) `class AppContext : EntityFrameworkDbContext`

10) Compléter le code suivant afin de persister les données ajoutées dans la base de données :

```
using (var context = new Context())
{
    context.Students.Add(new Student { Name = "Student 1" });
    context.SaveChanges()......................
}
```

Partie II : Étude de cas (12 pts)

La société tunisienne de l'électricité et de gaz (STEG) désire implémenter une application simplifiée de gestion des factures, à l'aide du Framework .Net 8 en suivant le diagramme de classes ci-dessous :



11) Implémenter le code de l'entité **Abonne** en incluant la ou les propriétés de navigation :

.....

.....

```
public class Abonne
{
    public string CIN {get; set;}
    public string Nom {get; set;}
    public string Prenom {get; set;}
    public IList<Compteur> Compteurs {get; set;}
}
```

12) Implémenter le code de **TypeCompteur** :

```
public enum TypeCompteur
{
    Domestique,
    Industriel,
    Autre
}
```

13) Sachant que la propriété **Reference** de **Compteur** est une clé primaire, compléter le code suivant :

```
public class Compteur
{
    [Key] ....
    public string Reference [illegible]
```

14) Sachant que la propriété **ConsommationKWH** de **Facture** est un nombre positif, compléter le code suivant en incluant un message en cas de violation de cette contrainte :

```
public class Facture
{
    [Range(0,int.MaxValue,ErrorMessage="ConsommationKWH doit être un nombre positif")]
    public int ConsommationKWH [illegible]
```

15) Sachant que la propriété **Debut** de **Periode** est seulement une date, compléter le code suivant :

```
public class Periode
{
    [DataType(DataType.Date)] ....
    public DateTime Debut [illegible]
```

16) On a ajouté au niveau de la classe de contexte toutes les entités, compléter le code pour l'entité **Abonne** sachant qu'elle doit être mappée à une table nommée **Abonnes** :

```
public class Context : [illegible]
{
    public DbSet<Abonne> Abonnes ............ [illegible]
```

17) On veut que la longueur de la colonne associée à la propriété **CIN** l'entité **Abonne** ne dépasse pas **150**, compléter le code suivant :

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity...<Abonne>().Property.......(e => e.CIN).HasMaxLength...(150);
```

18) En utilisant une classe de configuration, on veut établir la relation entre **Compteur**, **Periode** et **Facture** afin que les clés étrangères de cette relation soient mappées vers des colonnes nommées **CompteurKey** et **PeriodeKey**, compléter le code suivant :

```
internal class FactureConfiguration : IEntityTypeConfiguration<Facture...>
{
    public void Configure(EntityTypeBuilder<Facture...> builder)
    {
        builder.HasOne..(e => e.Compteur).
            WithMany.(e => e.Factures).HasForeignKey.....(e => e.CompteurKey);
        builder.HasOne..(e => e.Periode).
            WithMany.(e => e.Factures).HasForeignKey.....(e => e.PeriodeKey);
    }
}
```

19) La clé primaire de la classe **Facture** est composée des trois propriétés **CompteurKey**, **PeriodeKey** et **Date**, compléter le code suivant :

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity.............<Facture>().HasKey..(
    e => new {e.CompteurKey, e.PeriodeKey, e.Date});
```

20) On veut configurer toutes les propriétés de type **string** pour qu'elles soient mappées vers des colonnes de longueur maximale égale à **200**, choisir le bon emplacement :

☐	`protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)`
☐	`protected override void OnModelCreating(ModelBuilder modelBuilder)`
☒	`protected override void ConfigureConventions(ModelConfigurationBuilder configurationBuilder)`

☐ `protected override void HasConventions(ModelConfigurationBuilder configurationBuilder)`

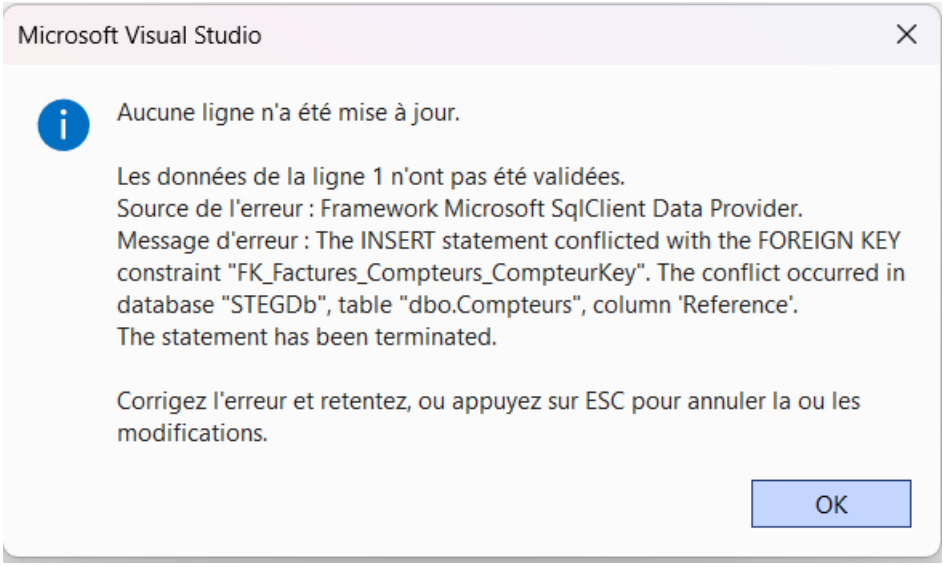
21) On veut avoir une base de données nommée **STEGDb**, quel est le bon emplacement au niveau de la chaine de connexion :

☐ 1	☐ 3
☒ 2	☐ 4

22) Une migration a été créée, écrire la commande qui permet de créer la base de données :

Update-Database .....

23) Après création de la base de données, on veut alimenter manuellement une ligne dans la table associée à l'entité **Facture**. On a eu ce message



Comment peut-on corriger cette erreur :

a) Supprimer la contrainte coté base de données
b) Forcer l'insertion
c) S'assurer qu'on a la clé dans la table associée à l'entité **Compteur**
d) Supprimer la ligne dans la table associée à l'entité **Computer**

24) En utilisant un code qui permet de charger la liste des compteurs, on a eu une exception de type **NullReferenceException**. Faire le nécessaire au niveau de la méthode appropriée pour ne pas avoir ce type d'exception :

```
protected override void OnConfiguring....(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder.
        UseLazyLoading....Proxies().UseSqlServer(
}
```

Bon travail ☺